Optimizing Language Model Pretraining on AWS Trainium 2: Microarchitectural Co-Design, Distributed Execution, and Custom Kernel Acceleration

GPU Proxy Ablation Dynamics and Empirical Findings

Maximizing validation bits-per-byte (val_bpb) under a rigid 30-minute wall-clock training budget requires disentangling algorithmic convergence efficiency from accelerator-specific execution artifacts[1]. Extensive proxy experiments conducted on NVIDIA RTX PRO 6000 Blackwell Max-Q (96GB HBM) and NVIDIA L4 (24GB VRAM) platforms establish an empirical baseline for transformer architecture, learning rate schedules, and dataloader binning before committing to AWS Trainium 2 silicon[2].

The experimental pretraining harness optimizes next-token prediction, evaluating validation performance using val_bpb [1]. Measuring bits-per-byte rather than standard cross-entropy token loss renders evaluation strictly invariant to vocabulary size modifications, ensuring that changes to tokenization, sequence packing, or structural embeddings remain directly comparable across runs[1]:

$$\text{val_bpb} = \frac{\mathcal{L}_{\text{val}}}{\ln(2)}$$

The initial reference architecture configured a 59.6-million parameter decoder-only transformer operating across context length $T = 1024$, hidden dimension $d = 640$, and depth $L = 10$, incorporating root-mean-square layer normalization (RMSNorm), rotary position embeddings (RoPE), query-key normalization (QK-norm), squared rectified linear activation (ReLU^2) multi-layer perceptrons, and logit softcapping[2].

Across an 800-step standardized evaluation unit, proxy experimentation examined twelve structural and procedural interventions, culminating in an 8.53% net relative improvement in val_bpb (lowering the score from an unmodified baseline of 1.273797 to 1.165273) through eight accumulated enhancements, while identifying and discarding four distinct structural regressions[2].

Phase /	Architec	Paramet	Step	Validatio	Relative	Selectio

Intervention	tural / Schedule Configuration	er Count	Latency (s)	n BPB	Delta	n Decision
Baseline (Solution 0)	Constant peak learning rate, warmup = 20 steps, packing factor = 1	59.6M	1.28	1.273797	Baseline	Reference Point[2]
Solution 1: LR Cooldown	Warmdown ratio = 0.75, final learning rate floor fraction = 0.05	59.6M	1.28	1.215996	-4.55%	Kept[2]
Solution 2: Warmup Removal	Warmup steps = 0 (full learning rate initialized at step 0)	59.6M	1.28	1.216120	+0.01%	Kept (Statistical Wash)[2]
Solution 3: Weight Decay Decay	Weight decay linearly decayed to 0 across training budget	59.6M	1.28	1.215870	-0.02%	Kept[2]

Solution 4: Muon Warmup Shift	Muon momentum warmup duration contracted from 300 to 200 steps	59.6M	1.28	1.216712	+0.07%	Discarded (Regression)[2]
Solution 5: Initialization Scaling	Global weight initialization standard deviation scaled by 0.68	59.6M	1.28	1.210940	-0.41%	Kept[2]
Solution 6: Packing Reshape (8x)	Packing bin multiplier = 8 (sequence length = 8192, reshaped to 1024)	59.6M	0.84	1.188680	-1.84%	Superseded[2]
Solution 6b: Packing Sweep (16x)	Packing bin multiplier = 16 (re-swept at full cumulative configuration)	85.9M	0.98	1.174631	+0.21%	Discarded (Leakage)[2]

Solution 6c: Packing Sweep (4x)	Packing bin multiplier = 4 (re-swept at full cumulative configuration)	85.9M	1.00	1.166012	-0.53%	Superseded[2]
Solution 6d: Packing Sweep (2x)	Packing bin multiplier = 2 (re-swept at full cumulative configuration)	85.9M	1.08	1.165273	-0.59%	Kept (Interior Optimum)[2]
Solution 6e: Packing Sweep (1x)	Packing bin multiplier = 1 (re-swept at full cumulative configuration)	85.9M	1.41	1.205512	+3.45%	Discarded (Crop Loss)[2]
Solution 7: Value Embeddings	Alternating layer injection with learned per-token gating	85.9M	0.85	1.174188	-1.22%	Kept[2]

Solution 8: Window Attention Bundle	Window pattern SSSSL, short fraction = 0.125, RoPE base = 200,000	85.9M	0.94	1.175473	+0.11%	Discarded (Regression)[2]
Sol 8 Decomp: RoPE Base Alone	RoPE base = 200,000 evaluated in isolation with full dense attention	85.9M	1.08	1.192130	+2.31%	Discarded (Regression)[2]
Sol 8 Decomp: Window Alone	Window pattern SSSSL evaluated in isolation on L4 proxy hardware	85.9M	4.56	1.295870	+2.59%	Discarded (Regression)[2]
Solution 9a: Local Causal Conv	Depthwise causal conv1d ($k =$) inserted before self-attention	85.9M	0.97	1.171962	-0.19%	Kept[2]
Solution	SwiGLU	102.3M	1.16	1.175858	+0.33%	Discarde

9b: Gated MLP (SwiGLU)	bilinear gating replacing squared ReLU in MLP blocks					d (Through put Drag)[2]
Solution 9c: Selective Gating	Dynamic input-de pendent gating on residual and skip scalars	85.9M	4.56	1.272345	+0.73%	Discarde d (Regressi on)[2]
Solution 10: Weight-C ast Cache	Cached BF16 linear projectio n weights across accumula tion inner loops	85.9M	0.97	1.172216	0.00%	Kept (Neutral / Overhea d-Free)[2]
Documen t Boundary Masking	Block-dia gonal attention mask preventin g cross-do cument attention	85.9M	1.08	1.171596	+0.54%	Discarde d (Regressi on)[2]
VE Weight Decay	Weight decay of 0.003 applied specifical	85.9M	4.56	1.264037	+0.07%	Discarde d (Flat / Marginal Drag)[2]

	ly to value embeddi ng tables					
Final Extended Run	Cumulati ve winning configura tion evaluated across 1450 training steps	85.9M	0.97	1.106820	-13.11%	Validated Baseline[2]

The single largest convergence accelerator emerged from the learning rate schedule redesign in Solution 1[2]. In a short, compute-constrained pretraining regime, maintaining peak learning rate across the entire run prevents weights from settling into narrow, low-entropy loss basins[2]. Transitioning from a flat schedule to an aggressive linear cooldown spanning the final 75% of total elapsed wall-clock time ($\text{WARMDOWN_RATIO} = 0.75$), while maintaining a small non-zero terminal floor equal to 5% of peak learning rate ($\text{FINAL_LR_FRAC} = 0.05$), produced a standalone -4.55% reduction in validation bits-per-byte[2].

Eliminating the standard 20-step learning rate warmup entirely (Solution 2) preserved initial optimization momentum, proving statistically neutral to slightly advantageous, while linear decay of weight decay to zero (Solution 3) prevented over-regularization during final convergence[2].

Conversely, reducing the Muon optimizer's momentum warmup from 300 steps down to 200 steps (Solution 4) caused an immediate +0.07% regression, demonstrating that the second-moment accumulator within Muon requires adequate step stabilization when paired with width-scaled learning rates[2].

The dataloader packing experiments (Solution 6) revealed critical interactions between sample truncation and cross-document attention leakage[2]. Conventional token packing crops documents that exceed row capacity ($T + 1$), discarding remaining document tails[3]. Expanding the bin size via reshaping ($\text{inputs.reshape}(\text{batch} \times k, T)$) directly reduces the frequency of tail-truncation events while cutting Python dispatch and gradient-accumulation iterations[2].

Sweeping the packing multiplier across values of 1, 2, 4, 8, and 16 identified an interior optimum

at $\mathrm{PACK_FACTOR} = 2$ [2]. At $\mathrm{PACK_FACTOR} = 1$, severe crop loss inflated $\mathrm{val_bpb}$ to 1.205512 (+3.45% regression)[2].
Conversely, expanding the bin factor to 16 concatenated numerous distinct document fragments into each row, introducing cross-document attention leakage that worsened $\mathrm{val_bpb}$ by +0.00241 relative to the 8x configuration[2]. Re-sweeping this parameter established $\mathrm{PACK_FACTOR} = 2$ as the Pareto frontier, capturing sufficient document length while preserving sequence coherence[2].
Subsequent attempts to implement dynamic document-boundary attention masking to mitigate leakage at large bin sizes failed (+0.00632 regression)[2]. In ultra-short training regimes, soft cross-document attention appears to serve as an implicit regularizer, introducing contextual entropy that prevents premature parameter collapse[2].
Architecturally, injecting learned Value Embeddings (VE) at alternating transformer layers (Solution 7) yielded a substantial -1.22% improvement[2]. By computing a dynamic gating vector:

$$g_v = 2 \cdot \sigma(\mathbf{W}_{\text{gate}} \mathbf{x}_{[:,:32]})$$

and modulating the value stream:

$$\mathbf{v} \leftarrow \mathbf{v} + g_v \odot \mathbf{E}_{\text{val}}$$

the network acquired 26.3 million parameters of position-specific memorization capacity without noticeably degrading execution step times[2].
This was complemented by Solution 9a, which inserted an independent depthwise causal 1D convolution ($k = 4$, zero-initialized) immediately preceding the multi-head attention block, capturing local inductive priors at linear computational cost and securing an additional -0.19% gain[2].
In contrast, structural modifications such as SwiGLU gated MLPs (Solution 9b) proved strictly detrimental under a fixed time budget[2]. While SwiGLU improves parameter capacity in unconstrained scaling, it added 16.4 million non-embedding parameters (+72% network parameter expansion) and inflated step execution latency by 19.2%, ultimately preventing the training loop from executing sufficient gradient descent steps to overcome the added capacity[2].
Similarly, sliding-window attention (Solution 8) was discarded: fine-grained decomposition revealed that windowing alone regressed by +2.59% because the local causal convolutions had already saturated local feature extraction, rendering the spatial truncation of self-attention purely destructive to global context processing[2].

AWS Trainium 2 Hardware Architecture and

NeuronCore-v3 Microarchitecture

Deploying these validated model components to AWS Trainium 2 requires understanding the specialized compute and memory layout of the silicon[6]. A single Trainium 2 accelerator integrates eight physical NeuronCore-v3 processing units fabricated on an advanced high-density process, delivering aggregate computational performance up to 667 dense TFLOPS of BF16/FP16/TF32 compute, 1,299 dense TFLOPS of FP8 compute, and 181 TFLOPS of standard FP32 operations[8].

The chip is backed by 96 GiB of HBM3 memory across four physical stacks, providing 2.9 TB/s of aggregate memory bandwidth[8]. Inter-accelerator scaling is facilitated by NeuronLink-v3 interconnects supplying 1.28 TB/s of bi-directional chip-to-chip bandwidth organized in a low-latency 2D-torus topology[8].

Architectural Dimension	**Trainium 1 (Trn1)**	**Trainium 2 (Trn2)**	**NVIDIA H100 SXM5**
Compute Engines per Chip	2 NeuronCore-v2	8 NeuronCore-v3	132 SMs
Dense BF16 Compute (TFLOPS)	190	667	989
Dense FP8 Compute (TFLOPS)	Not Supported	1,299	1,979
Standard FP32 Compute (TFLOPS)	48	181	67
On-Chip Scratchpad / Cache	48 MiB SBUF (24 MiB/core)	224 MiB SBUF (28 MiB/core)	50 MiB L2 Cache
Dedicated Accumulator SRAM	4 MiB PSUM (2 MiB/core)	16 MiB PSUM (2 MiB/core)	Register File / Shared Memory
Device Memory Technology	32 GiB HBM2e	96 GiB HBM3	80 GiB HBM3
Memory Bandwidth (TB/s)	0.82	2.90	3.35

Direct Memory Access Bandwidth	~1.2 TB/s	3.5 TB/s	N/A (Cache Managed)
Inter-Chip Interconnect	NeuronLink-v2 (800 GB/s)	NeuronLink-v3 (1,280 GB/s)	NVLink 4.0 (900 GB/s)
Thermal Design Power (TDP)	~300 W	~700 W	700 W

The NeuronCore-v3 internal architecture departs fundamentally from conventional GPU Symmetric Multiprocessor (SM) designs[12]. Unlike GPUs that rely on hardware-managed, hierarchical L1/L2 caches to buffer memory accesses dynamically, NeuronCore-v3 exposes completely software-managed, explicitly addressed on-chip SRAM[13].

The primary memory tier within each core is the State Buffer (SBUF), providing 28 MiB of SRAM per core (up from 24 MiB in Trainium 1's NeuronCore-v2), which yields 224 MiB of collective on-chip scratchpad capacity across the 8-core accelerator[8].

SBUF operates at approximately 20 times the bandwidth of external HBM3[13]. Complementing SBUF is the Partial Sum Buffer (PSUM), a specialized 2 MiB SRAM array strictly dedicated to accumulating matrix multiplication results emitted by the systolic processing arrays[13].

Both SBUF and PSUM are organized as 2D memory arrays governed by the Neuron Instruction Set Architecture (ISA) 128-partition constraint[14]. The first dimension of any memory tile—the Partition Dimension (P)—is physically mapped across 128 parallel hardware execution lanes and cannot exceed a size of 128[14].

The second dimension—the Free Dimension (F)—contains contiguous elements processed per partition[14]. In SBUF, each partition encompasses 224 KiB of storage, while a PSUM partition contains 16 KiB[15]. If a computation specifies $P < 128$, the unused hardware lanes idle, directly degrading functional compute efficiency[18].

Within each NeuronCore-v3, computation is distributed across five specialized heterogeneous execution engines that operate concurrently[12]:

The Tensor Engine (TensorE) is centered around a 128×128 systolic processing array that executes dense matrix multiplications, tensor contractions, and spatial layout convolutions[12]. TensorE reads stationary matrices and moving matrices exclusively from SBUF and accumulates outputs directly into PSUM[12]. It supports native FP8 (E4M3, E5M2), BF16, FP16, TF32, and FP32 datatypes[8]. NeuronCore-v3 introduces built-in hardware transpose acceleration within TensorE, delivering a 2x throughput uplift for FP32 transposition and direct BF16 PSUM emission, bypassing older emulation routines[15].

The Vector Engine (VectorE) is a wide SIMD array optimized for reductions, normalizations (LayerNorm, RMSNorm), spatial pooling, and vector-tensor elementwise operations[12]. Arithmetic executes internally in FP32 precision with zero-overhead casting to and from 16-bit representations[16]. NeuronCore-v3 introduces a high-performance mode that shares a wide physical memory bus with the GPSIMD engine, unlocking a 2x to 4x speedup on select BF16/FP16 vector transforms[12].

The Scalar Engine (ScalarE) is dedicated to nonlinear pointwise mathematical evaluations, such as Gaussian Error Linear Units (GELU), exponentiations, reciprocals, square roots, and combined scale-bias affine projections[12]. ScalarE possesses deeply pipelined arithmetic logic units that can fuse scaling, biasing, and transcendental activation evaluation into a single execution instruction[18].

The General Purpose SIMD Engine (GPSIMD) comprises eight independent, fully programmable 512-bit vector processing cores executing custom C/C++ routines[12]. GPSIMD handles dynamic control flow, custom random number generation (such as Philox RNG algorithms for stochastic operations), and non-standard indexing patterns directly on-chip without stalling TensorE[12].

The Sync Engine (SyncE) serves as the hardware orchestration unit that tracks instruction dependencies across all engines[16]. Synchronization is enforced via 32-bit hardware monotonic semaphores[19]. Rather than executing costly global barriers, engines issue instructions tagged with semaphore wait conditions ($\text{sem} \geq \text{threshold}$) and increment conditions upon retirement ($\text{sem} += 1$), achieving microsecond-level synchronization across pipeline stages[19].

Data movement between external HBM3 and on-chip SBUF is governed by the Direct Memory Access (DMA) engine, which supports an aggregate bandwidth of 3.5 TB/s per chip and includes inline hardware memory compression/decompression units[8].

However, achieving high effective memory bandwidth utilization (MBU) requires structuring DMA transfers to avoid the Small DMA Trap[18]. When memory accesses use small, non-contiguous strides, the DMA engine incurs severe transaction descriptor setup overhead, causing effective bandwidth to plummet while compute engines stall on memory wait semaphores[18].

Empirical profiling on Trainium indicates that individual DMA packet payloads must exceed 32 KiB to saturate physical channel throughput[18]. For an FP32 tile spanning 128 partitions, the free dimension must satisfy:

$$F \geq \frac{32768 \text{ bytes}}{128 \text{ partitions} \times 4 \text{ bytes/element}} = 64 \text{ elements}$$

while BF16 transfers require $F \geq 128$ elements to surpass the 32 KiB threshold[18].

AWS Neuron SDK Compilation Dynamics and Runtime

Optimization

Executing pretraining workloads on AWS Trainium 2 requires navigating the compilation lifecycle managed by the AWS Neuron SDK (specifically torch-neuronx and neuronx-cc)[6].
Standard PyTorch code is lowered via OpenXLA into High-Level Optimizer (HLO) computation graphs, which the Neuron compiler optimizes through layout assignment, loop tiling, and operator fusion before compiling into Neuron Executable File Format (NEFF) binaries[6].
A central imperative of the Neuron runtime is strict adherence to static tensor shapes[3]. The Neuron compiler operates Ahead-of-Time (AOT)[21]. If the training harness encounters a dynamic shape variation—such as uneven sequence length padding, variable dataloader batch slices, or re-dimensioned intermediate activations—the execution environment triggers an immediate Just-in-Time (JIT) recompilation stall[3].
Each JIT recompilation event forces a compile cycle that halts execution for 60 to 180 seconds[3]. Within a fixed 30-minute competition window, two or three unintended recompilation events can consume 15% to 20% of the entire training budget, irreparably degrading cumulative step count and final $\mathrm{val_bpb}$ [2].
Consequently, all sequence packing dimensions, model hyperparameters, and batch tensor structures must remain fixed across the initialization, training, and validation cycles[2].
To ensure that the initial training steps are not penalized by graph generation latencies, all execution graphs must be precompiled prior to initiating the timed run[6]. The competition harness incorporates an official startup allowance excluding compilation time from the charged 30-minute window, designating the first five optimizer iterations as the startup boundary[2].
To comply with the automated submission checkers, the training script must invoke the boundary tracking callback prepare.record_step_boundary(step) at the completion of every optimizer iteration, as failure to issue this hook triggers automatic disqualification[2].
Graph compilation must be executed via neuron_parallel_compile, an orchestration CLI tool that traces the model graph across all execution ranks simultaneously and executes multi-threaded compiler lowerings using local host CPUs[8].
The resulting NEFF artifacts must be persisted to a fast, locally attached NVMe storage volume via the environment configuration NEURON_COMPILE_CACHE_DIR=/tmp/neuron_cache[6].
Storing the persistent compiler cache on shared remote network file systems (such as Amazon FSx for Lustre) must be avoided[6].
As established during the large-scale HLAT pretraining runs on Trainium clusters, concurrent read/write lock contention against identical cached graph structures across multiple ranks over network links induces severe communication timeouts and runtime stalls[6]. Localizing the cache completely eliminates host I/O bottlenecks during initialization[6].
Optimal hardware compilation requires passing specialized flags to neuronx-cc to enable aggressive graph transformations suited for decoder transformers[6]. The flag --distribution-strategy=llm-training directs the compiler to reorder parameter updates, optimize gradient accumulation buffers, and schedule inter-core collective communication primitives across data-parallel workers to maximize compute and communication overlap[6].

Passing --model-type=transformer triggers pattern-matching passes that identify canonical Multi-Head Attention and MLP structures, automatically fusing linear projections and matching attention mechanisms to native Tensor Engine primitives[6].
The target execution topology is explicitly passed using --lnc=1 (or --lnc=2) to align compilation with the active Logical NeuronCore runtime configuration[10].
Furthermore, passing --internal-hlo2tensorizer-options='--fast-relayout=true' instructs the backend code generator to implement tensor transpositions via Tensor Engine matrix multiplications using identity matrices, achieving a 2x speedup over software-emulated permutation loops[15].
At runtime, performance is further governed by two critical environment variables: NEURON_FUSE_SOFTMAX=1 and NEURON_RT_ASYNC_EXEC_MAX_INFLIGHT_REQUESTS=3[6].
Setting NEURON_FUSE_SOFTMAX=1 forces the lowering engine to collapse the three-stage Softmax pipeline (maximum subtraction, exponential calculation, reciprocal sum normalization) into a single fused execution kernel within the Vector and Scalar Engines, preventing intermediate activation spills into external HBM[6].
Configuring NEURON_RT_ASYNC_EXEC_MAX_INFLIGHT_REQUESTS=3 establishes an asynchronous execution pipeline between the host CPU runtime and the NeuronCore hardware queue[6]. This depth of three allows the host to construct and dispatch subsequent micro-batch execution requests while the current micro-batch is processing on the silicon, hiding host dispatch overhead and eliminating inter-step pipeline bubbles[6].

Distributed Execution and Precision Strategies for 8 NeuronCores

In Phase 1, computational resources consist of a single physical Trainium 2 accelerator containing eight NeuronCore-v3 processing units[4]. Maximizing training throughput under this configuration requires careful selection of parallel distribution topologies, logical core mappings, and precision modes[6].
The fundamental architecture enables configuring the hardware through the Logical NeuronCore (LNC) setting, controlled via NEURON_LOGICAL_NC_CONFIG[10]. When set to LNC=1, the runtime exposes all eight physical cores as discrete execution ranks (NC_v3_0 through NC_v3_7)[10]. Each rank possesses 12 GiB of dedicated HBM3 capacity and 28 MiB of SBUF[8].
When configured as LNC=2 (the Trn2 platform default), adjacent physical cores are combined in pairs into four logical execution units (NC_v3d_0 through NC_v3d_3)[10]. In this mode, each logical rank controls 24 GiB of unified HBM address space and 56 MiB of aggregate SBUF, facilitating intra-pair collective exchanges without accessing external network fabric[10].

Operational Parameter	Logical Core Mode: LNC = 1	Logical Core Mode: LNC = 2 (Default)

Exposed Logical Ranks	8 Independent Ranks (NC_v3_0 to NC_v3_7)	4 Unified Ranks (NC_v3d_0 to NC_v3d_3)
Physical Cores per Rank	1 Core	2 Cores
Addressable HBM per Rank	12 GiB Dedicated HBM3	24 GiB Shared HBM3
Addressable SBUF per Rank	28 MiB	56 MiB Aggregate
Optimal Parallelism Mapping	Pure Data Parallelism ($DP = 8$)	Data Parallel ($DP = 4$) or Hybrid ($TP = 2, DP = 2$)
Inter-Core Communication	Gradient AllReduce across 8 on-chip cores	Intra-pair shared memory collectives
Target Model Scale Fit	Models $\leq 100\text{M}$ parameters (High step rate)	Models $> 500\text{M}$ parameters (Memory bound)

For a pretraining model scaled to 60–85 million parameters, pure Data Parallelism ($DP = 8$) under LNC=1 is architecturally superior to Tensor Parallelism (TP) or Pipeline Parallelism (PP)[2].
Tensor Parallelism shards individual weight matrices across execution ranks, requiring two high-frequency AllReduce communication barriers per transformer layer (one for multi-head attention, one for the MLP)[6]. At an 85M model dimension ($d = 640$), sharding across 8 tensor-parallel ranks yields an inner projection tile width of:

$$\frac{640}{8} = 80 \text{ channels}$$

This severely under-utilizes the 128-element systolic arrays of TensorE and introduces fine-grained communication latency on every forward and backward pass[16].
Conversely, pure Data Parallelism replicates the parameters across all eight cores, requiring zero inter-core synchronization during forward and backward passes[6]. Synchronization is

restricted to a single gradient AllReduce executed once per optimizer step across the 8 on-chip cores[6].

With 16 dedicated Collective Communication Cores (CC-Cores) orchestrating data transfers over the ultra-wide on-chip fabric, the gradient all-reduce latency is negligible compared to the compute phase[8].

Memory optimization across the eight data-parallel workers is achieved using ZeRO-1 (Zero Redundancy Optimizer Stage 1) sharding[6]. In ZeRO-1, model weights and first-order gradients remain local to each rank, but the optimizer states (the FP32 master weights, momentum buffers, and second-moment accumulators of AdamW and Muon) are partitioned equally across the 8 ranks[6].

For an 85M model using Muon and AdamW, complete optimizer state allocation consumes approximately 1.0 to 1.4 GB per rank, which easily fits within the 12 GiB boundary of LNC=1, leaving over 10 GB of HBM entirely free for activation buffering and dynamic sequence batches[2].

Hardware-accelerated numerical precision is a major differentiator of the Neuron architecture[6]. Conventional GPU training relies on mixed precision, keeping FP32 master weights and casting weights dynamically to BF16 for forward and backward passes to prevent catastrophic rounding stalls[2].

Trainium 2 implements hardware Stochastic Rounding (SR), enabled globally via NEURON_RT_STOCHASTIC_ROUNDING_EN=1[12]. Unlike deterministic Round Nearest Even (RNE), which rounds numbers to the nearest representable floating-point bit, stochastic rounding rounds values probabilistically proportional to their proximity to the adjacent representations[12]:

$$P(\mathrm{SR}(x) = \lceil x \rceil) = \frac{x - \lfloor x \rfloor}{\lceil x \rceil - \lfloor x \rfloor}$$

Mathematically, stochastic rounding is an unbiased estimator:

$$\mathbb{E}[\mathrm{SR}(x)] = x$$

As proven by Connolly, Higham, and Mary (2021), this property replaces Wilkinson's classical deterministic worst-case error accumulation bound of $\mathcal{O}(nu)$ with a probabilistic mean-zero bound of $\mathcal{O}(\sqrt{n}u)$, where u represents unit roundoff[12].

Consequently, minute gradient updates that fall below the precision epsilon of a standard 7-bit BF16 mantissa are not systematically truncated to zero[12]. This enables pure BF16 storage and execution across model weights, activations, and optimizer updates without requiring expensive FP32 master weight retention, cutting memory footprint and memory-bus traffic by nearly half[6].

However, incorporating stochastic rounding alters AdamW optimization stability[6]. As demonstrated during the HLAT pretraining ablations, setting the second-moment decay

parameter to $\beta_2 = 0.99$ causes optimization instability under stochastic rounding[3]. Because stochastic rounding introduces high-frequency, zero-mean variance into individual gradient calculations, an overly long exponential moving average window ($\beta_2 = 0.99$) responds too slowly to transient gradient spikes, causing the running variance to lag behind rapid parameter movements[6].

Decreasing the parameter to $\beta_2 = 0.95$ contracts the effective historical smoothing window, allowing the optimizer to adapt quickly to gradient fluctuations and maintain numerical stability throughout pretraining[3].

Custom Kernel Optimization and Operator Fusion via NKI

When PyTorch operators execute in eager mode on Trainium, each distinct function call incurs host dispatch overhead and forces intermediate tensors to materialize in external HBM3[3].

The Neuron Kernel Interface (NKI) provides a Python-embedded programming framework providing explicit, low-level control over NeuronCore ISA instructions, engine scheduling, and SBUF/PSUM on-chip allocation[14]. Writing custom NKI kernels eliminates HBM round-trips by fusing multiple operations into single-pass execution tiles held entirely within SBUF[17].

The primary candidate for operator fusion is RMSNorm, which executes approximately 42 times per forward pass in a 10-layer transformer[3]. Standard PyTorch dispatches three isolated operations: mean-square reduction along the channel dimension, reciprocal square root (rsqrt), and weight multiplication[3].

An NKI fused RMSNorm kernel binds the sequence token dimension across the 128 SBUF partitions ($P \leq 128$) and maps the feature dimension $d = 640$ to the free axis (F)[3].

VectorE computes the squared summation across F using nisa.tensor_reduce, ScalarE executes the rsqrt via its pipelined reciprocal-root hardware functional unit, and VectorE completes the affine projection in SBUF before streaming results back to HBM[16].

This single-pass execution eliminates three memory round-trips per call, saving over 120 memory barrier events per step[3].

A secondary target is Rotary Position Embeddings (RoPE), which dispatches 20 times per forward pass[3]. The canonical implementation segments queries and keys into halves, negates the secondary half, multiplies both segments by cosine and sine frequencies, and concatenates the results via torch.cat[2].

The concatenation operation allocates and populates a new tensor in HBM[3].

An NKI RoPE kernel tiles queries and keys directly into SBUF, maps head dimensions to the free axis, applies in-place rotation using VectorE instructions, and writes the transformed data back via non-blocking DMA, eliminating buffer allocation overhead[17].

A tertiary target is Logit Softcapping, which applies an elementwise bounded hyperbolic tangent transformation across raw vocabulary predictions:

$$\text{logits}_{\text{capped}} = C \cdot \tanh\left(\frac{\text{logits}}{C}\right)$$

where $C = 30.0$ [2].

Because this operation targets the prediction tensor of shape $[B, T, V]$—where vocabulary size $V \approx 8192$ to 32768 represents the widest dimension in the network—materializing the softcapped tensor in HBM incurs substantial memory bandwidth penalties[2].

An NKI kernel fuses softcapping directly into the cross-entropy loss computation[3].

Tiling the vocabulary across SBUF chunks of size 128×512, ScalarE applies the scaled $\tanh$ activation, passes the output to VectorE to accumulate the running online log-sum-exp denominator, and stores only the scalar cross-entropy loss to HBM[3]. The massive $[B, T, V]$ intermediate tensor is never materialized in external memory[3].

To maximize the efficiency of these custom kernels, their design must be aligned with the microarchitectural execution rules identified in hardware profiling[18]. Profiling tools like neuron-profile report spill_save_bytes and spill_reload_bytes to measure data transferred between SBUF and HBM due to on-chip capacity exhaustion[18].

If an NKI kernel allocates large buffers inside outer loops, the compiler may insert automated spill instructions that degrade arithmetic intensity[18]. Declaring local allocation buffers strictly within the innermost loop scopes ensures that physical SBUF partition addresses are recycled immediately after intermediate consumption, eliminating memory thrashing[18].

DMA transfer sizing must similarly be respected[18]. As established in microarchitectural benchmarks, the DMA engine suffers severe latency penalties when issuing small transfers ($< 32\ \text{KiB}$)[18]. In NKI, data loading via nl.load and storage via nl.store must tile contiguous multidimensional chunks that maximize both partition ($P = 128$) and free dimensions ($F \geq 128$ for BF16)[14].

For operations on shapes smaller than 128 channels, partition vectorization must be applied: multiple logical vectors must be packed into disjoint partition slices of a single 128-wide SBUF tile, allowing a single vector instruction to process multiple items in parallel[18].

Furthermore, intermediate Partition-Free (PF) transpositions can be eliminated through operator co-design[18]. When the layout of a tensor emitted into PSUM does not match the input layout required by a downstream engine, the compiler inserts a PF-transpose that consumes TensorE execution cycles[18].

In a Linear-to-RMSNorm pipeline, mapping the weight tensor of the projection to the moving matrix input and the activation feature map to the stationary matrix swaps the row-column emission layout in PSUM[18]. This produces activations pre-aligned with VectorE reduction

requirements, removing the intermediate PF-transpose instruction entirely[18].
Finally, dense attention can be accelerated via a custom NKI FlashAttention kernel executing in a single pass without materializing the full $T \times T$ attention matrix in HBM[3]. Sequence length $T = 1024$ is decomposed into chunks of 128 tokens, matching the physical partition lanes[17]. The kernel iterates across the key and value blocks, computing partial dot products $\mathbf{QK}^T$ using TensorE[14].
To compute the correct global Softmax denominator incrementally, the Vector and Scalar engines execute the Online Softmax algorithm[17]. When processing block j with local row maximum $m^{(j)}$, the global maximum is updated:

$$m_{\text{new}} = \max(m_{\text{old}}, m^{(j)})$$

The prior accumulator values are updated using a dynamic rescaling factor:

$$\alpha = e^{m_{\text{old}} - m_{\text{new}}}$$

The running sum of exponentials updates as:

$$L_{\text{new}} = \alpha \cdot L_{\text{old}} + \sum \exp(S^{(j)} - m_{\text{new}})$$

and the running attention-weighted value accumulator updates as:

$$\mathbf{O}_{\text{new}} = \alpha \cdot \mathbf{O}_{\text{old}} + \exp(S^{(j)} - m_{\text{new}})\mathbf{V}^{(j)}$$

Double-buffered asynchronous DMA transfers (nl.load) prefetch the subsequent $\mathbf{K}^{(j+1)}$ and $\mathbf{V}^{(j+1)}$ tiles from HBM into alternate SBUF banks while TensorE multiplies the current block, overlapping memory transfers and arithmetic execution[17].

Fixed Wall-Clock Time Budget Pretraining Dynamics

Pretraining a language model under a fixed wall-clock budget ($T_{\text{budget}} = 1800$ seconds) alters optimal scaling relationships relative to compute-optimal Chinchilla formulations[1]. In standard regimes, compute is treated as a static allocation of total FLOPs:

$$C \approx 6ND$$

where parameters N and dataset tokens D scale in balanced proportion[34].
In a fixed-time competition, the total token exposure is not fixed; it is an endogenous variable

governed by physical hardware throughput[2]:

$$D_{\text{total}} = \tau(N) \cdot T_{\text{budget}}$$

where $\tau(N)$ represents the sustained token throughput per second as a function of model parameterization N [2].

This dynamic penalizes over-parameterized models[2]. If depth (L) or width (d) is expanded excessively, per-step execution latency increases[2].
Because the training duration is fixed at 1800 seconds, fewer total optimizer updates are completed[2].
As observed in the GPU proxy runs, scaling depth to 11 and hidden dimension to 640 reduced total step count, preventing the optimizer from navigating past early plateaus and resulting in higher final validation loss[2].
Conversely, an under-parameterized model processes tokens rapidly but suffers from a restricted representational capacity, hitting an irreducible validation entropy floor early in the run[2].
The optimal balance requires selecting an architecture that maximizes parameter capacity while preserving execution efficiency on the systolic arrays[2].
The baseline architecture uses an aspect ratio:

$$\frac{d}{L} = 64$$

with $L = 10$ and $d = 640$ [2].
Because 640 is an exact integer multiple of the 128-partition hardware dimension ($640 = 5 \times 128$), the projection weights saturate the physical execution lanes of the Tensor Engine without requiring zero-padding[2].
Integrating Value Embeddings increases effective parameter capacity to 85.9 million parameters (+44% parameter scale) while operating at zero additional FLOP cost during the multi-head attention matrix multiplications[2].
Similarly, the local causal convolution ($k = 4$) adds an informative structural inductive bias at linear computational cost $\mathcal{O}(T \cdot d)$, avoiding the quadratic scaling $\mathcal{O}(T^2 \cdot d)$ of attention[2].
Data consumption efficiency is governed by the dataloader packing strategy[2]. The optimal configuration uses online CPU tokenization and sequence packing running concurrently with on-chip execution[6].
As established in the empirical ablations, configuring a packing factor of

$\mathrm{PACK_FACTOR} = 2$ (yielding an input batch shape of $[B \cdot 2, T]$) minimizes document cropping while bounding cross-document attention leakage, sustaining optimal token throughput without host-side stalls[2].

Prioritized Enhancement Roadmap for Phase 1

The operational roadmap for Phase 1 on a single Trainium 2 accelerator structures interventions into three sequential, risk-mitigated phases: zero-risk algorithmic modifications, distributed runtime orchestration, and profiling-guided custom kernel synthesis[2].

Phase 1A incorporates validated algorithmic adjustments directly into train.py, carrying zero device-porting risk and introducing no shape alterations that could trigger compiler graph mutations[2].

The learning rate warmdown schedule is configured to decay linearly across the final 75% of the 30-minute budget ($\mathrm{WARMDOWN_RATIO} = 0.75$), computing decay progress from elapsed wall-clock seconds rather than step iterations to absorb initial compile delays, while terminating at a non-zero floor ($\mathrm{FINAL_LR_FRAC} = 0.05$)[2].

The initial warmup is eliminated entirely ($\mathrm{WARMUP_STEPS} = 0$), and weight decay is set to decay linearly to zero in synchronization with the learning rate cooldown[2].

Non-projection weight initialization standard deviations are scaled by 0.68[2].

Dataloader sequence packing is set to a factor of 2, reshaping incoming $2 \times T$ buffers into contiguous rows of length $T = 1024$ [2].

Alternating-layer Value Embeddings with dynamic gating and the zero-initialized local causal convolution ($k = 4$) are enabled in the model architecture[2].

Finally, linear projection weights are cached in BF16 format after each optimizer step to eliminate redundant casting during gradient accumulation loops[2].

Phase 1B focuses on the distributed execution setup across the eight NeuronCores and the configuration of the Neuron runtime environment[6].

The execution environment is configured for LNC=1 via NEURON_LOGICAL_NC_CONFIG=1, launching eight discrete ranks via torchrun --nproc_per_node=8 train.py to establish pure Data Parallelism ($DP = 8$)[10].

ZeRO-1 optimizer state sharding is enabled across the ranks, partitioning AdamW and Muon states to preserve local HBM[6].

Stochastic Rounding is enabled globally via NEURON_RT_STOCHASTIC_ROUNDING_EN=1, allowing the entire network state, gradients, and optimizer updates to remain in pure BF16[12].

To stabilize optimization under stochastic rounding noise, the AdamW second-moment decay is configured to $\beta_2 = 0.95$ with $\beta_1 = 0.90$ [3].

Compilation flags are set to --distribution-strategy=llm-training --model-type=transformer

--lnc=1 --internal-hlo2tensorizer-options='--fast-relayout=true', while runtime environment variables are configured with NEURON_FUSE_SOFTMAX=1 and NEURON_RT_ASYNC_EXEC_MAX_INFLIGHT_REQUESTS=3[6].
The persistent cache directory NEURON_COMPILE_CACHE_DIR is targeted to local NVMe storage, and neuron_parallel_compile is executed across all ranks to compile all computation graphs before beginning the timed competition run[6].
Phase 1C addresses low-level hardware kernel optimization guided by performance profiling[3]. Initial steady-state training iterations are traced using neuron-profile to inspect compute engine utilization, SBUF spilling metrics (spill_save_bytes), and DMA wait latencies[18].
Based on the execution trace, custom NKI kernels are implemented and deployed in order of impact: first, a fused single-pass RMSNorm kernel operating across 128 partitions; second, an in-place fused RoPE kernel eliminating intermediate slice and concatenation materializations; and third, a fused Logit Softcap and Cross-Entropy kernel that pipelines hyperbolic tangent bounds directly into online log-sum-exp accumulation[3].
Before committing kernels to scored runs, framework equivalence tests and the submission causality check (prepare.causality_check) are executed to ensure strict mathematical equivalence and prevent silent gradient corruption[2].

Milestone	Optimization Strategy	Implementation Level	Expected Mechanism / Gain	Risk Level	Dependencies
Phase 1A	LR Warmdown (0.75, floor 0.05)	Python / Schedule	Optimal late-stage convergence (-4.55% bpb)	Zero	None[2]
Phase 1A	Warmup Elimination (0 steps)	Python / Schedule	Maximizes steps at full learning rate	Zero	None[2]
Phase 1A	Linear Weight Decay Decay	Python / Schedule	Prevents late-stage over-regularization	Zero	None[2]
Phase 1A	Initialization Scale (0.68)	Python / Model Init	Improves initial	Zero	None[2]

			gradient propagation		
Phase 1A	Packing Reshape (Factor 2)	Dataloader / Reshape	Eliminates crop loss; limits leakage	Low	Static Shape[2]
Phase 1A	Value Embeddings (Alternating)	PyTorch Architecture	Adds 26.3M params at near-zero FLOP cost	Low	Memory Budget[2]
Phase 1A	Local Causal Conv ($k =$)	PyTorch Architecture	Captures local sequence patterns cheaply	Low	Zero Init[2]
Phase 1A	Weight-Cast Caching	PyTorch Linear	Cuts redundant HBM weight casts	Zero	None[2]
Phase 1B	Logical Core Config (LNC=1)	Runtime / Compiler Flag	Exposes 8 discrete data-parallel ranks	Low	Neuron SDK[10]
Phase 1B	Pure Data Parallelism ($DP =$)	Distributed Harness	Eliminates inter-layer TP barriers	Low	LNC=1 [cite: 6, 11]
Phase 1B	ZeRO-1 Optimizer Sharding	Neuron Distributed	Shards optimizer state across 8 cores	Low	$DP =$ [cite: 6, 7]

Phase 1B	Stochastic Rounding Mode	Hardware ISA / Runtime	Unbiased BF16 updates; cuts HBM traffic	Low	$\beta_2 = 0.95$ [cite: 6, 12]
Phase 1B	Compiler Flags & Async Queue	Compiler Flags / Env	Graph optimizations and overlapped dispatch	Low	Static Shapes[6]
Phase 1B	Local AOT Precompilation	CLI / NVMe Storage	Eliminates JIT compilation runtime stalls	Zero	prepare.py [cite: 2, 6]
Phase 1C	Execution Trace Profiling	neuron-profile GUI	Identifies engine bottlenecks and spills	Zero	Training Run[18]
Phase 1C	NKI Fused RMSNorm	NKI Python / SBUF	Fuses 42 calls/step into single SBUF pass	Medium	Vector/ScalarE[3]
Phase 1C	NKI Fused RoPE	NKI Python / SBUF	Removes torch.cat buffer allocation	Medium	VectorE[3]
Phase 1C	NKI Fused Softcap + CE	NKI Python / SBUF	Avoids materializing $[B, T, V]$ in HBM	High	Scalar/VectorE[3]

Phase 1C	NKI FlashAttention Kernel	NKI Python / Systolic	Single-pass attention with online softmax	High	Double Buffering[17]

Strategic Synthesis

Maximizing pretraining efficiency within a fixed 30-minute allocation on a single Trainium 2 accelerator requires treating the model, training schedule, and hardware execution pipeline as an integrated system[2].

Algorithmic refinements validated through GPU proxy testing provide an immediate, low-risk foundation: aggressive learning rate warmdown with a small residual floor, early warmup elimination, decaying weight decay, and scaled initialization establish strong convergence dynamics[2].

Dataloader packing tuned to an interior optimum ($\mathrm{PACK_FACTOR} = 2$) significantly mitigates sample truncation while avoiding the cross-document attention leakage that degrades larger bin configurations[2].

Architecturally, injecting capacity through alternating-layer Value Embeddings and local causal convolutions expands representational expressiveness at near-zero computational overhead, outperforming over-parameterized alternatives such as SwiGLU gated MLPs that inflate step latency and limit total optimization iterations[2].

Realizing these convergence gains on Trainium 2 silicon depends on matching model execution to the underlying NeuronCore-v3 microarchitecture[6]. Operating across eight discrete data-parallel ranks under an LNC=1 topology avoids the fine-grained collective communication overhead of tensor parallelism, while ZeRO-1 optimizer sharding distributes state memory across cores with minimal synchronization cost[6].

Hardware-accelerated stochastic rounding enables end-to-end BF16 execution without numerical vanishing gradients, provided the AdamW second-moment decay is tuned to $\beta_2 = 0.95$ to accommodate stochastic variance[6].

Rigidly maintaining static tensor dimensions and precompiling execution graphs via neuron_parallel_compile into local NVMe storage safeguards the fixed wall-clock budget against Just-in-Time recompilation penalties[3].

Finally, targeted NKI operator fusion across high-frequency memory-bound primitives—specifically RMSNorm, RoPE, and logit softcapping—eliminates intermediate HBM roundtrips by constraining multi-stage computations to on-chip SBUF partitions[3].

Systematically executing this phased roadmap maximizes training throughput and drives validation bits-per-byte to its theoretical minimum within the allotted competition window[2].

Works cited

1. Karpathy Autoresearch Complete 2026 Guide.pdf

2. gpu-experiment-results.md
3. enhancement-opportunities.md
4. AWS Trainium Frontier competition: Co-design models and kernels, https://www.amazon.science/news/aws-trainium-frontier-competition-co-design-models-and-kernels-on-purpose-built-ai-chips
5. Karpathy Left His GPU Running Overnight.pdf
6. HLAT High-quality LLM Pre-trained on AWS Trainium 2404.10630v2.pdf
7. AWS Trainium2: Scalable ML Accelerator, https://www.emergentmind.com/topics/aws-trainium2
8. Trainium2 Architecture - AWS Neuron Documentation - Read the Docs, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/arch/neuron-hardware/trainium2.html
9. What Is AWS Trainium? Amazon's AI Chip & PCB Design ... - NextPCB, https://www.nextpcb.com/blog/what-is-aws-trainium-ai-chip-pcb
10. Logical NeuronCore configuration - AWS Neuron Documentation, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/arch/neuron-features/logical-neuroncore-config.html
11. What is Neuron Collective Communication?, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/neuron-runtime/about/collectives.html
12. Trainium positioning - trnsci, https://trnsci.dev/trainium_positioning/
13. The Trainium Memory Hierarchy - AWS Neuron Documentation, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/get-started/about/memory-hierarchy-overview.html
14. GitHub - stanford-cs149/asst4-trainium2, https://github.com/stanford-cs149/asst4-trainium2
15. Trainium2 Architecture Guide for NKI - AWS Neuron Documentation, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/guides/architecture/trainium2_arch.html
16. Trainium/Inferentia2 Architecture Guide for NKI, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/guides/architecture/trainium_inferentia2_arch.html
17. Deep Dive into FlashAttention on AWS Neuron NKI.pdf
18. NKI Performance Optimizations.pdf
19. A Barrier-Free Synchronization Algorithm for Multi-Engine AI ... - arXiv, https://arxiv.org/html/2608.13757v1
20. GPU vs TPU vs ASIC vs FPGA: Which AI Chip Dominates in 2027?, https://www.nextpcb.com/blog/gpu-vs-tpu-vs-asic-vs-fpga-ai-chip
21. Mastering Inference on Specialized Silicon Neuron and TPU.pdf
22. NKI Performance Optimizations - AWS Neuron Documentation, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/deep-dives/nki_perf_guide.html
23. Programming AWS Trainium with the Neuron Kernel Interface, https://hidekazu-konishi.com/entry/programming_aws_trainium_with_the_neuron_kernel_interface.html

24. Train Llama2 with AWS Trainium on Amazon EKS.pdf
25. Neuron Compiler CLI Reference Guide (neuronx-cc), https://awsdocs-neuron.readthedocs-hosted.com/en/latest/compiler/neuronx-cc/api-reference-guide/index.html
26. Hugging Face BERT Pretraining Tutorial (Data-Parallel), https://awsdocs-neuron.readthedocs-hosted.com/en/v2.27.0/frameworks/torch/torch-neuronx/tutorials/training/bert.html
27. Training Llama-3.1-70B and Llama-3-70B with Tensor Parallelism, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/libraries/neuronx-distributed/tutorials/training_llama_tp_pp.html
28. Amazon EC2 Trn2 instances and UltraServers - AWS, https://aws.amazon.com/ec2/instance-types/trn2/
29. aws-neuron-samples/torch-neuronx/inference ... - GitHub, https://github.com/aws-neuron/aws-neuron-samples/blob/master/torch-neuronx/inference/hf_pretrained_pixart_sigma_1k/neuron_pixart_sigma/compile_transformer_latency_optimized.py
30. Neuron compiler CLI Reference Guide (neuron-cc), https://awsdocs-neuron.readthedocs-hosted.com/en/latest/compiler/neuron-cc/command-line-reference.html
31. Neuron Device Memory, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/neuron-runtime/explore/device-memory.html
32. Training with Neuron - FAQ, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/faq/training/neuron-training.html
33. Neuron Rounding Modes, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/arch/neuron-features/rounding-modes.html
34. We brought Mamba to Trainium — and mapped where custom, https://lab.cloud/research/custom-kernels-on-trainium/
35. Distributed Training with optimum-neuron - Hugging Face, https://huggingface.co/docs/optimum-neuron/en/guides/distributed_training